

DINO-WM on PointMaze under a Commodity-GPU Budget: Latent-Cached Reproduction, Low-Data Transfer, and Planning Results

Technical report — method, implementation, and results

Thomas Georges*

June 2026 — v1.0 (50-evaluation results; 200-evaluation pass pending)

Abstract

We reproduce DINO-WM [1] — an action-conditioned latent transition model trained over frozen DINOv2 patch features for zero-shot visual planning — on the PointMaze task, under the constraint that the full experiment must run on freely available Colab hardware. The stock training loop required more than 30 hours per epoch on this hardware, independent of batch size; we identify the cause as per-sample episode reload compounded by online re-encoding of the *frozen* encoder, and replace it with a precomputed-latent training path that reproduces the upstream data distribution bit-for-bit while cutting per-step cost by $\approx 21\times$ (58 minutes per epoch on a T4, an estimated 3–6 on an A100). On this foundation we run three experiments: a full-data predictor (2000 rollouts), a low-data scratch baseline (300 rollouts), and a low-data fine-tune initialized from the full-data checkpoint. The fine-tuned predictor reaches the scratch run’s converged latent-prediction loss at its *first* epoch ($\sim 30\times$ fewer updates), and in closed-loop CEM planning in the real MuJoCo environment it delivers $2.75\times$ the scratch baseline’s success rate (0.275 vs 0.10 at 200 evaluation episodes, $p < 10^{-5}$) while *matching* the full-data model trained on $6.7\times$ more data (0.275 vs 0.23, $p=0.30$). A striking dissociation emerges: the scratch and fine-tune predictors land close in validation loss yet differ by $2.75\times$ in planning success, evidence that latent-prediction MSE is a weak proxy for downstream control. All numbers are read from recorded planner telemetry (the latest `final_eval` per run, at 200 evaluation episodes). An addendum (§11) reuses this checkpoint as the trusted global model for a coupled local/global planning study over six planners: a global re-score gate makes cheap first-order refinement *safe*, recovering global-CEM-quality plans (0.97 success) where an unguarded hybrid collapses to 0.54, while the purely local planners run 100–1000 $\times$ cheaper per episode.

1 Introduction

A world model is useful for control to the extent that planning against it produces actions that succeed in the real environment. DINO-WM [1] makes a specific bet about how to obtain such a model cheaply: rather than learning perception and dynamics jointly, it *freezes* a powerful self-supervised visual encoder (DINOv2 [2]) and learns only an action-conditioned transition model over its patch features, then plans by sampling-based optimization of latent goal distance. The appeal is data- and compute-efficiency — no pixel reconstruction is needed in the planning loop, and the encoder is reused unchanged across tasks.

*Implementation assistance: Claude (Anthropic). Code: `wm-prediction` repository, `src/wm_poc/dino_wm`. This report has an addendum, *Local/Global World-Model Planning with DINO-WM* (`reports/local_global_report.tex`), which builds a local/global planning track on the checkpoint and cache documented here.

This report documents a faithful reproduction of that pipeline on PointMaze, developed under a hard practical constraint: it must run end to end on the hardware actually available to us (Colab, predominantly T4; A100 rarely). That constraint turned out to be the technically interesting part. The stock pipeline does not merely run slowly on a T4 — it is structurally infeasible, projecting more than 30 hours per epoch regardless of batch size. Diagnosing and removing that cost *without* altering the experiment is the methodological contribution of §5; it is what makes the results of §9 obtainable at all.

Contributions. (i) A latent-cache training path for DINO-WM that removes the dominant data-path cost while provably preserving the upstream training distribution (§5); (ii) a faithful, fully resumable reproduction of DINO-WM predictor training and CEM planning on PointMaze, runnable on commodity GPUs (§7); (iii) a low-data transfer study — scratch vs. fine-tune from a full-data checkpoint — with both latent-prediction and real-environment planning metrics (§9); and (iv) the observation that latent-prediction loss and planning success dissociate sharply, which we argue is a methodological warning for world-model evaluation (§10). This reproduction is also the foundation for the coupled local/global planning study of §11, which reuses this checkpoint and cache unchanged as a trusted, non-differentiated global model.

2 Background

DINO-WM. DINO-WM [1] encodes each frame to frozen DINOv2 patch tokens $\mathbf{z}_t \in \mathbb{R}^{P \times D}$ and trains a transformer predictor to advance the latent state conditioned on actions, with a latent-consistency objective and no decoder required for planning. Planning is visual goal reaching: given current and goal observations, sampling-based MPC minimizes the predicted latent distance to the goal encoding. The model’s defining property for our purposes is that the encoder is frozen — its outputs are constants of the dataset, a fact the training loop should but does not exploit.

Self-supervised features for control. The broader pattern of planning on top of frozen self-supervised representations (DINOv2 [2]; the JEPa world-model family [6]) trades end-to-end representation learning for modularity and sample efficiency. DINO-WM is a concrete, planning-focused instance.

Sampling-based latent planning. PlaNet [3] and Dreamer [4] plan or learn policies inside a learned recurrent latent model; TD-MPC2 [5] pairs a learned latent model with sampling-based MPC. DINO-WM keeps the model fixed (frozen encoder, separately trained predictor) and plans with the cross-entropy method (CEM), which requires only forward rollouts and is the planner reproduced here.

3 Problem setting and notation

PointMaze episodes are sequences of frames with raw 2-D actions $\mathbf{a}_t \in \mathbb{R}^2$, bounds $[-1, 1]^2$, and proprioception (agent position/velocity). Following DINO-WM, one *model step* advances f raw steps (frameskip $f=5$); the model-step action is the folded block $\bar{\mathbf{a}}_k = [\mathbf{a}_{kf}; \dots; \mathbf{a}_{(k+1)f-1}] \in \mathbb{R}^{fA}$. Each frame is encoded to $\mathbf{z} \in \mathbb{R}^{P \times D}$ with $P=196$ patches and $D=384$ (DINOv2 ViT-S/14 at 224px, whose internal resize yields a 14×14 patch grid). The predictor consumes $H=3$ context frames (`num_hist`) and predicts the next latent (`num_pred=1`), trained autoregressively over windows. Episode-level train/validation splits are drawn from a seeded permutation of the uncapped episode count, so split membership is invariant to how many rollouts a given run uses.

4 The DINO-WM model as reproduced

Encoder (frozen). `dinov2_vits14`, feature key `x_norm_patchtokens`, weights frozen throughout (`train_encoder=false`). Input frames are preprocessed with the upstream transform (resize/centre-crop to 224px, normalize), then resized to the 196-px encoder input that produces the 14×14 patch grid.

Predictor. A ViT predictor (`models.vit.ViTPredictor`; depth 6, 16 heads, MLP width 2048, dropout 0.1) over the flattened $H \times P = 3 \times 196 = 588$ token sequence. Actions and proprioception are embedded (`action_emb=proprio_emb=10`) and, with `concat_dim=1`, concatenated along the feature axis (tiled across patches), giving the predictor working dimension $D + 10 + 10 = 404$ (observed as “Model emb.dim: 404” in the run logs). A causal mask is applied across the temporal axis.

Training objective. With $\hat{\mathbf{z}}$ the predicted next latent and $\mathbf{z}$ the (detached) target encoding,

$$\mathcal{L} = \|\hat{\mathbf{z}} - \text{sg}[\mathbf{z}]\|_2^2, \quad (1)$$

the latent-consistency MSE, reported per epoch as a training and a validation component (the “h-step” validation loss). The runs in this report are *decoder-free* (`has_decoder=false`): no pixel reconstruction is trained, which both halves memory and matches the planning objective, since planning never decodes. The action and proprioception encoders are trained jointly with the predictor; the DINOv2 encoder is shared and frozen by construction.

5 Feasible training: the latent-cache path

5.1 Why the stock loop costs 30 hours per epoch

Profiling the upstream data path identified three compounding causes, none of which batch size can address (confirmed empirically: batch 4 and batch 32 both project $\sim 19\text{--}21$ h/epoch on a T4).

1. **Per-sample episode reload.** Upstream `TrajSlicerDataset` fetches a 4-frame window by `torch.load`-ing the *entire* episode image tensor (~ 93 frames) and transforming *every* frame, then discarding all but four. At $\sim 147\text{k}$ windows/epoch this re-reads and re-transforms $\sim 1600 \times$ the dataset per epoch, through the Google Drive FUSE mount.
2. **Online re-encoding of frozen features.** With `train_encoder=false` the DINOv2 outputs are dataset constants, yet every frame of every window is re-encoded every epoch — pure recomputation.
3. **Operational confounders.** Single-threaded loading (`num_workers=0`, an OOM-era setting) and reads from Drive rather than local disk.

5.2 Encode once, reuse forever

Because the encoder is frozen, its outputs can be precomputed. Three components, installed into the upstream checkout at launch time as marker-guarded patches:

1. **One-time precompute.** Each episode is read once, preprocessed *exactly* as upstream does, encoded under autocast through the frozen encoder, and written as a per-episode `fp16 .npy` plus a manifest (encoder identity, patch geometry, episode lengths, dataset coverage). The cache is ~ 30 GB for all 2200 episodes, held on session-local disk (a Drive-hosted cache

would reinstate the FUSE bottleneck; the scripts warn if so configured), and the process is idempotent and resumable.

2. **A drop-in latent dataset**, selected purely by Hydra override. It subclasses the upstream dataset, so actions, states, proprioception, normalization statistics, and the seeded train/val split are *bit-identical* to the online path (verified directly against the upstream loader); its slicer memory-maps only the ~ 0.6 MB of latents each window needs instead of materializing a ~ 13 MB episode.
3. **An input-dispatch bypass** in the model’s `encode_obs`: 4-D latent inputs skip the encoder; 5-D image inputs take the original path. The real DINOv2 weights remain in the model and its checkpoints, so planning (which encodes real observations) is untouched.

This is an *exact* reproduction, not an approximation: the only quantities that change are when and how often the frozen encoder runs. A guard rejects combining the latent path with the (pixel-target) decoder at command-build time.

5.3 Throughput

Configuration	Steps/epoch	Step time (T4)	Epoch (T4)	Epoch (A100, est.)
Stock, batch 4	36,815	~ 3 s	> 30 h	—
Stock, batch 32	4,602	15.9–17 s	19–21 h (proj.)	—
Latent cache, batch 32, stride 1	4,602	~ 0.76 s	~ 58 min	6–12 min
Latent cache, batch 32, stride 2	2,330	0.72–0.76 s	31 min (meas.)	3–6 min

Table 1: Training throughput before and after the latent-cache path. The residual step cost is the depth-6 ViT predictor itself (~ 2 TFLOPs fwd+bwd at batch 32 over 588 tokens) — the quantity the experiment is meant to measure. “Stride 2” subsamples window start positions (consecutive stride-1 windows overlap by 19/20 frames) and was used only for the full run on T4; the low-data pair uses the full stride-1 window set.

6 Planning and evaluation

Planning uses the upstream CEM [8]/MPC path unchanged. Given a start and a goal observation, CEM optimizes an open-loop action sequence to minimize the predicted latent distance to the goal encoding:

$$J(\bar{\mathbf{a}}_{1:H}) = \frac{1}{PD} \|\hat{\mathbf{z}}_H(\bar{\mathbf{a}}_{1:H}) - \mathbf{z}_{\text{goal}}\|_2^2, \quad (2)$$

iterating a sample/score/refit loop (population 100, 10 iterations, goal horizon $G=5$, `goal_source=random.stat`). Crucially, while CEM *optimizes* an imagined latent quantity, **success is measured in the real MuJoCo environment**: the planned actions are executed in the simulator and success is the environment’s own goal-reaching test. Reported success rates are therefore real closed-loop control outcomes, not the model’s self-assessment. Each evaluation also records, per CEM iteration, the current best plan’s environment success, mean state distance to goal, and the divergence between the imagined and executed latent trajectories (the model’s own rollout error).

7 Implementation

The pipeline follows a two-notebook convention: an operational notebook (02) for setup, pre-compute, training, fine-tuning, and planning — every heavy step delegated to a gated CLI

script against a *pristine* upstream checkout — and a read-only results notebook (07) that aggregates artifacts into tables, curves, and embedded planning videos. Reproducibility and reliability features:

- **Pristine upstream, runtime patches.** All behavioral changes (mixed precision, latent cache, validation `no_grad`, fine-tune initialization, decoder-free video recording) are marker-guarded, idempotent, reversible textual patches applied at launch, with backups; the upstream repository is never vendored or edited in place. Patch application and composition are unit-tested against the real upstream source.
- **Single experiment, hardware-only profiles.** GPU-tier configs differ only in throughput knobs (precision, dataloader workers, wall-clock caps, and — on T4 only — window stride); model sizes, optimizer budgets, episode counts, and run names are shared so a run continues across tiers.
- **Resumability.** Full epoch checkpoints plus a rolling intra-epoch state-dict checkpoint (every 2000 steps on A100, 500 on T4) with deterministic mid-epoch resume bound a disconnection to minutes of lost work.
- **Self-gating cells.** Each launch cell checks durable state before spending GPU time: completed training is skipped, partial training resumes, and planning re-triggers only when the recorded evaluation does not match the configured episode count — making a top-to-bottom run idempotent.
- **Smoke crash test.** A 20-rollout configuration exercises the exact production pipeline (patches, precompute, latent training, checkpointing, a 2-episode planning pass) in ~ 10 –15 minutes; it caught a planner/latent-dataset incompatibility, a checkpoint resolution bug, and a blank-checkpoint crash before any cost full-run time.
- **Testing.** ~ 95 unit tests (runnable without torch) cover patch application/composition, command rendering, run-state gates, and metrics extraction; latent-dataset parity against the upstream loader is validated separately with torch.

8 Experimental setup

Component	Setting	Component	Setting
Encoder	DINOv2 ViT-S/14, frozen, 224 px	Predictor	ViT, depth 6, 16 heads, MLP 2048
Latents	$P=196$, $D=384$, fp16 cache	Working dim	404 (384+10+10, concat)
Context / pred	$H=3$, <code>num_pred=1</code>	Frameskip	$f=5$
Batch size	32	Predictor lr	5×10^{-5} (scratch)
Action-enc. lr	5×10^{-4} (scratch)	Fine-tune lr	10^{-5} / 10^{-4}
CEM pop. / iters	100 / 10	Goal horizon G	5
Goal source	<code>random_state</code>	Planning evals	200

Table 2: Configuration. Precision is bf16 on A100/L4 and fp16 on T4 (Turing emulates bf16 slowly); all runs are seed 0.

Three runs enter the results (Table 3). All train the predictor only, on cached latents, with identical seeds, normalization, and splits.

Run	Data (train+val)	Epochs	Initialization	Planning evals
Full no-decoder	2000 + 200	10	random	200
Low-data scratch	300 + 60	30	random	200
Low-data fine-tune	300 + 60	20	full checkpoint	200

Table 3: Runs. Fine-tuning loads the predictor and action/proprio encoders from the full run’s checkpoint and continues at reduced learning rates; the frozen DINOv2 encoder is shared. The full run was trained on a T4 with stride-2 window subsampling (Table 1).

9 Results

All numbers below are read directly from recorded artifacts — the *latest final_eval* row of each run’s planner `logs.json`, at 200 evaluation episodes per checkpoint (Wilson half-width ≈ 0.06).

9.1 Training loss: fine-tuning starts where scratch ends

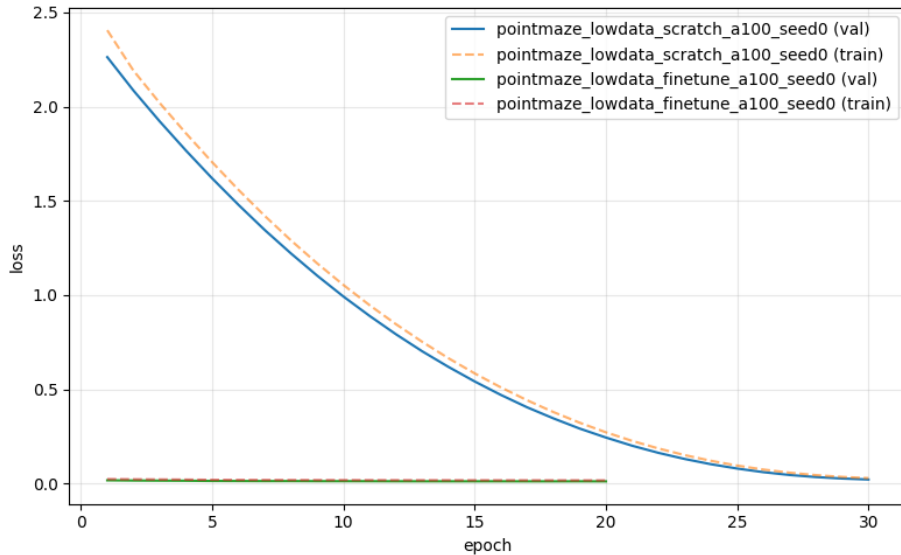


Figure 1: Per-epoch training (dashed) and validation (solid) latent-prediction loss. The low-data scratch run descends from ≈ 2.3 to ≈ 0.02 over 30 epochs; the fine-tune begins at ≈ 0.015 at epoch 1 — already the scratch run’s converged level — and stays there for all 20 epochs.

The fine-tuned predictor attains at *epoch 1* the validation loss the scratch baseline reaches only after ~ 30 epochs: pre-training delivers the converged predictor for $\sim 30\times$ fewer updates on the low-data task. The honest caveat is that scratch *does* converge to the same validation loss by epoch 30 (Figure 2). On the latent-prediction objective alone, the two models therefore converge to the same order of magnitude — which makes the planning result that follows all the more pointed.

9.2 Planning success in the real environment

At $n=200$ the Wilson intervals tighten to ± 0.06 , and two clean results stand out. **Fine-tune** $>$ **scratch** is decisive (0.275 vs 0.10 ; two-proportion $z=4.5$, $p < 10^{-5}$): pre-training on the full corpus and adapting on 300 rollouts yields $2.75\times$ the planning success of training on those same 300 rollouts from scratch. **Fine-tune vs. full-data** (0.275 vs 0.23) is statistically

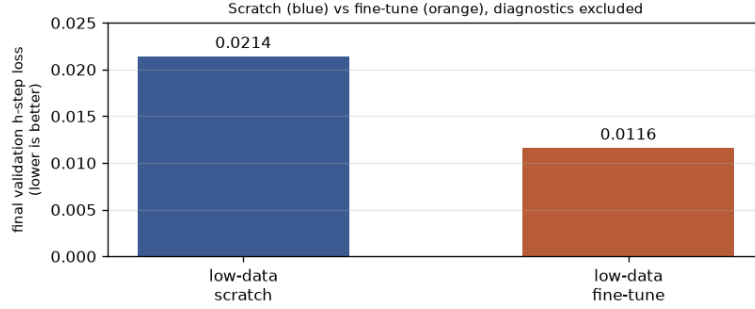


Figure 2: Final validation h-step loss, scratch vs. fine-tune: ≈ 0.021 vs ≈ 0.012 — the same order of magnitude, in sharp contrast to their $2.75\times$ gap in planning success (Figure 3).

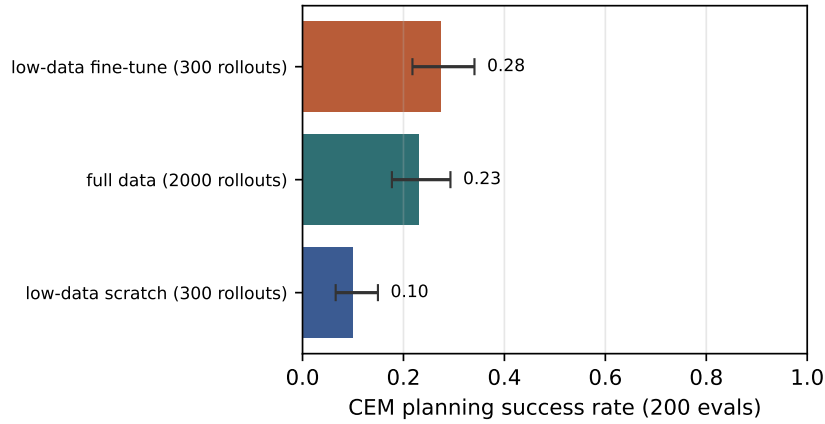


Figure 3: Closed-loop CEM planning success in MuJoCo PointMaze, 200 evaluation episodes per checkpoint, with 95% Wilson intervals. Exact values from the planner logs: fine-tune 0.275, full-data 0.23, scratch 0.10.

indistinguishable ($p=0.30$): fine-tuning *matches* a model trained on $6.7\times$ more data. The full-data model’s 0.23 is moreover a lower bound: it was trained on a T4 with stride-2 window subsampling (half the gradient updates per epoch of the paper-exact stride-1 set), so an A100 stride-1 training of the identical configuration should only improve it — which would make the fine-tune’s parity with it the more striking. (The larger sample also moved all three estimates down from the noisier 50-eval interim, $0.34/0.26/0.12 \rightarrow 0.275/0.23/0.10$, without disturbing the ranking or either conclusion.)

9.3 The loss/planning dissociation

The central empirical finding is the contrast between Figures 2 and 3: scratch and fine-tune converge to the same order of magnitude in validation latent-prediction loss (≈ 0.021 vs ≈ 0.012) yet differ $2.75\times$ in real-environment planning success (0.10 vs 0.275 , $p < 10^{-5}$). The $\sim 2\times$ residual loss gap is far too small to explain a near-tripling of control success: latent-MSE convergence does not imply equal control quality — the fine-tuned model’s latent geometry is evidently more *plannable* (smoother or better-calibrated for CEM’s optimization) despite matching the scratch model’s average prediction error. The practical consequence for world-model evaluation is direct: a planning metric must be carried alongside the loss curve; the loss alone would have reported these two models as equivalent.

9.4 Planner diagnostics: CEM is genuinely optimizing

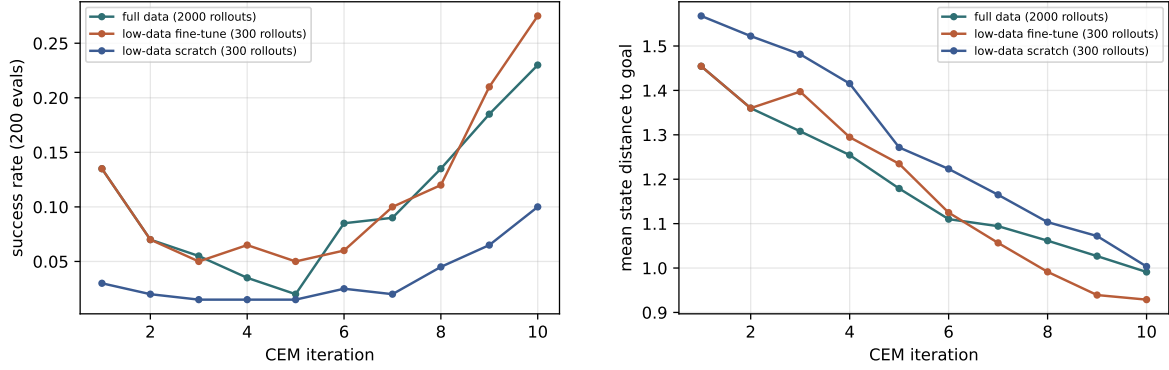


Figure 4: Planner-internal telemetry per CEM iteration (exact values from `logs.json`; for relaunched evaluations the latest attempt per iteration is shown). Left: environment success of the current best plan rises across the 10 iterations (full $0.135 \rightarrow 0.23$; fine-tune $0.135 \rightarrow 0.275$; scratch $0.03 \rightarrow 0.10$). Right: mean state distance to goal falls correspondingly (e.g. fine-tune $1.45 \rightarrow 0.93$).

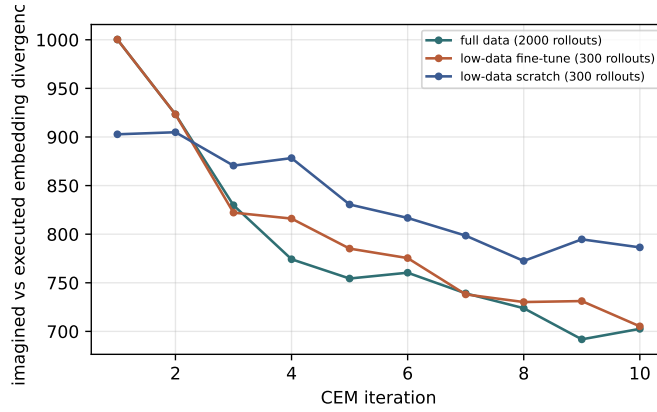


Figure 5: Divergence between imagined and executed visual embeddings per CEM iteration — the world model’s own rollout error along the optimized plan. The fine-tune and full models keep imagination closest to reality at the end of optimization.

The iteration curves confirm CEM is exploiting the learned dynamics rather than stumbling onto goals: success rises and goal distance falls monotonically over the 10 CEM iterations for every trained model, with the scratch model improving least — consistent with its weaker, less plannable dynamics. The embedding-divergence curve (Figure 5) shows the better models also keep imagination closer to executed reality, which is exactly the property a planner needs.

9.5 Planning videos

Each evaluation records per-episode MP4s (`output_final.<idx>.<success|failure>.mp4`; 26 frames at 12 fps, the $5 \times 5 + 1$ -step horizon) of the executed plan in the environment against the goal image. (A patch was required for this: upstream records videos only for decoder-bearing models, so the decoder-free checkpoints produced none until the recorder was extended with a no-decoder branch — executed rollout vs. goal, imagination panel blacked out. Videos predating the patch come only from a 1-epoch decoder-bearing smoke model and are not representative.)

10 Discussion and limitations

What these results establish. A faithful DINO-WM PointMaze reproduction is feasible on commodity hardware once the frozen-encoder cost is removed, and on it, transfer from a full-data checkpoint yields a real, measurable planning advantage on a low-data task — with the efficiency bonus that the fine-tuned predictor is converged after a single epoch. The loss/planning dissociation is, to us, the most scientifically interesting output and the firmest methodological lesson.

Threats to validity. (i) *Sample size*: 200 planning episodes (± 0.06 Wilson half-width); both headline comparisons clear the noise, and the fine-tune-vs-full parity is a genuine null rather than an under-powered one. (ii) *Single seed*: all runs are seed 0; a second seed of the low-data pair would strengthen the transfer claim. (iii) *Full-run stride*: the full model used T4 stride-2 subsampling, so its 0.23 is a conservative lower bound and cross-comparisons against it should note this. (iv) *Precision*: T4 runs use fp16, A100 bf16/TF32; results across tiers are statistically equivalent, not bit-identical. (v) A log-rotation flaw (since fixed) destroyed the full run’s early-epoch loss lines, so its training curve may begin after epoch 1; the model and all planning results are unaffected.

11 Addendum: Coupled Local/Global Planning

The trained DINO-WM checkpoint above is also the *global* model for a second study: cheap first-order (gradient-based) planning against a trusted, non-differentiated world model [7]. Zero-order planners (CEM) work with any accurate forward model but scale poorly in horizon and action dimension; first-order planners are sample-efficient per iteration but need a differentiable model — and a model small enough to differentiate cheaply is usually inaccurate enough to be *exploited*, with gradient descent walking into regions where it is wrong. The coupled local/global recipe decouples the two roles: the large model produces trajectories and is never differentiated, while a lightweight *local* surrogate supplies derivatives evaluated *along the global model’s trajectories*. We instantiate it on the PointMaze DINO-WM model and ask whether first-order refinement through a cheap surrogate can recover global-CEM-quality plans at a fraction of the global-model compute. The surrogate is trained in notebook 03 and the planners evaluated in notebook 08; the full method and protocol are in the companion file `reports/local_global_report.tex`.

11.1 Method

The **global model** is the frozen DINO-WM latent predictor above, used as forward simulator and scorer under `no_grad`. The **local model** is a small residual-MLP surrogate over a frozen orthogonal projection of the same latents, differentiable in the actions and trained on cached transitions (20k steps). Six **planners** are compared: `global_cem` (zero-order on the global model); `local_cem`, `local_gd`, `local_adam` (zero/first-order on the surrogate alone, isolating model error from optimizer effects); `hybrid_cem_local_refine` (global CEM, then local first-order refinement of the winning sequence); and `hybrid_cem_local_refine_global_rescore` (the same, but the global model re-scores the refinement and keeps it only if it improves the global objective). Three safeguards against surrogate exploitation are built in — the frozen orthogonal projection (anti-collapse), an action-smoothness penalty, and the global re-score gate — and the local-global *disagreement* (surrogate prediction vs. the global model’s executed step) is logged throughout.

11.2 Protocol and its stated bias

Planners are scored on offline latent goal reaching over 100 held-out episodes: normalized final distance $\rho = \|\mathbf{z}_{\text{fin}} - \mathbf{z}_{\text{goal}}\|^2 / \|\mathbf{z}_{\text{start}} - \mathbf{z}_{\text{goal}}\|^2$ and success $= \mathbb{I}[\rho < 0.5]$. **Because the global model is both simulator and judge**, `global_cem` is structurally favored on distance — it optimizes exactly the quantity it is scored on — so the distance/success columns establish only that the other planners are *competent*, while the efficiency columns (wall time, forward/backward call counts) and the acceptance rate carry the comparative claim. A *reference replay* of the dataset’s true action sequence through the same simulator calibrates the global model’s own rollout error at $\rho_{\text{ref}} = 0.058$.

11.3 Surrogate training

The surrogate reaches a final validation one-step loss of 0.0058 and a K -step rollout MSE of 0.0029 at 20,000 steps (per-horizon curve in Figure 6).

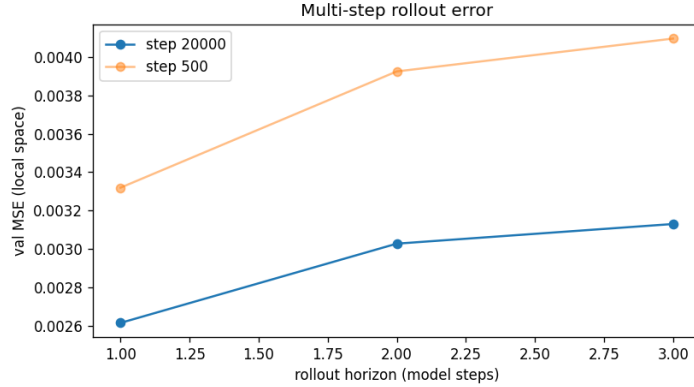


Figure 6: Surrogate validation rollout error vs. horizon (PointMaze, seed 0), exported by notebook 08 to the Drive `_summary` directory.

11.4 Planner comparison

Planner	succ.	ρ	wall/ep. (s)	glob. fwd	loc. fwd	bwd	accept	disagr.
<code>global_cem</code>	0.97	0.138	231.6	1.50M	—	—	—	—
<code>local_cem</code>	0.45	0.675	0.22	—	1.50M	—	—	0.0020
<code>local_gd</code>	0.13	1.306	2.60	—	50k	50k	—	0.0028
<code>local_adam</code>	0.41	0.699	2.67	—	50k	50k	—	0.0019
<code>hybrid_cem_local_refine</code>	0.54	0.632	234.1	1.50M	50k	50k	1.00	0.0019
<code>hybrid..._global_rescore</code>	0.97	0.138	234.2	1.50M	50k	50k	0.028	0.0024
reference replay	—	0.058	—	—	—	—	—	—

Table 4: Planner comparison on offline latent goal reaching, 100 held-out PointMaze episodes (`pointmaze_local_full_seed0`, surrogate at 20k steps). Distances are judged by the global model (see the stated bias), so the efficiency columns carry the comparative claim. **succ.**=success rate; ρ =normalized final distance; **fwd/bwd**=forward/backward call counts; **accept**=refinement acceptance rate; **disagr.**=mean local–global disagreement.

11.5 Findings

1. **Global CEM tops the distance metric** (0.97 success, $\rho = 0.138$), as the judge bias predicts; even so, its 0.138 is $\approx 2.4\times$ the reference-replay floor (0.058) — the global model

does not perfectly close its own gap.

2. **Pure local planners are far cheaper but weaker** on the global-judged metric: `local_cem` 0.45 at 0.22 s/episode, `local_adam` 0.41 at 2.7s, `local_gd` 0.13 — against `global_cem`’s 231 s/episode. The surrogate forward is $\sim 1000\times$ cheaper per call, but at this capacity it is not accurate enough to plan to global quality on its own.
3. **The unguarded hybrid hurts.** `hybrid_cem_local_refine` accepts *every* refinement (accept=1.00) and falls to 0.54 ($\rho = 0.632$): refining on the surrogate walks a good global plan into regions where the surrogate is wrong — exactly the exploitation failure mode the safeguards target.
4. **The global re-score gate fixes it.** `hybrid...global_rescore` rejects 97.2% of refinements (accept=0.028) and recovers global-CEM quality *exactly* (0.97, $\rho = 0.138$). This is the headline result: the gate turns a harmful refinement into a no-op, so the hybrid is never worse than the CEM it began from.
5. **Disagreement is small, yet gradients are unreliable.** The local–global disagreement stays ≈ 0.002 — the surrogate tracks the global model’s one-step behavior — yet its gradient *directions* improve the global objective rarely enough that the gate accepts only 2.8% of them. One-step tracking accuracy and gradient usefulness are not the same thing.

11.6 Synthetic sanity check (preliminary)

On the exact synthetic point-mass task (a few hundred surrogate steps; 2–3 episodes, a correctness signal rather than statistics) the expected ordering holds: all six planners succeed, `local_adam` attains the lowest final distance, the hybrids improve on plain CEM with their refinements accepted, and `local_gd` is weakest. In an adversarial unit test where the surrogate’s action gain is sign-flipped, refinement reliably worsens the global cost and the re-score gate rejects it, returning the CEM sequence unchanged.

11.7 What this establishes

The protocol measures planner behavior *under the global model’s belief*. It shows that first-order refinement through a cheap surrogate can be made *safe* — never worse than CEM — by a global re-score gate, and that the pure local planners are 100–1000 $\times$ cheaper per episode than the global baseline. It cannot, by construction, rank planners against reality; real-environment confirmation in MuJoCo PointMaze is the headline next step (§13). Results are single-seed (seed0), single-task.

12 Computational resources

A standing constraint of this project is that every experiment fit inside free-tier Colab limits, and that the experiment *definition* (model size, data budgets, episode counts) be identical across GPU tiers — only throughput knobs (precision, dataloader workers, micro-batch size) differ between a T4, an L4, and an A100 (§7). The figures below therefore describe a pipeline that runs end-to-end on free hardware, not a cluster job.

Hardware. Baseline runs use a free-tier NVIDIA T4 (16 GB, fp16); an A100-40 GB or L4 (bf16) is used opportunistically when Colab grants one. Turing emulates bf16 slowly, so the T4 runs in fp16 while A100/L4 use bf16; results across tiers are statistically equivalent, not bit-identical.

The enabling optimization. The stock DINO-WM loop re-encodes $\sim 1600\times$ the dataset through the frozen DINOv2 encoder every epoch, projecting 19–30 h per epoch on a T4 and putting the full study out of reach on free hardware. Precomputing the frozen patch latents once and training the predictor on that cache cuts per-step cost $\approx 21\times$ (Table 1): a measured 31 min/epoch on a T4 at stride 2, or 3–6 min on an A100. The cache is ~ 30 GB for all 2,200 episodes and lives on session-local disk.

Stage	Budget	Wall-clock
Latent precompute (one-time)	2,200 episodes $\rightarrow$ DINOv2	amortized; ~ 30 GB cache
Predictor training, full	10 epochs, 2,000+200 traj.	~ 31 min/epoch T4; 3–6 min A100
Predictor training, low-data ($\times 2$)	20–30 epochs, 300+60 traj.	well under the full run (15% data)
Planning eval (per checkpoint)	200 episodes, CEM 100×10	~ 47 min (A100, measured)
Surrogate training (§11)	20,000 steps	~ 1 – 2.5 h (T4)
Six-planner eval (§11)	100 episodes $\times$ 6 planners	~ 20 h total (see below)

Table 5: Wall-clock footprint of both studies on the stated hardware. Training and planning are *per checkpoint*; three checkpoints (full, low-data scratch, low-data fine-tune) enter the DINO-WM results.

Where the time goes. The six-planner sweep is dominated by the three planners that call the global model: each runs ≈ 232 – 234 s per episode (≈ 6.5 h for 100 episodes), against ≈ 0.2 – 2.7 s per episode — seconds to a few minutes in total — for the three purely local planners, a $\sim 1000\times$ per-call gap. This asymmetry is exactly the motivation for the coupled local/global design (§11): the global rollout is the dominant cost, so a cheap differentiable surrogate that substitutes for some of those rollouts is where any compute saving must come from.

Resumability. Because several stages exceed a single free-tier session — the six-planner sweep alone is ~ 20 h — every long-running stage is checkpointed and resumable: training writes per-epoch checkpoints, planning resumes per episode (the latest `final_eval` is authoritative), and the local/global evaluation runs under a 300-minute per-session wall cap with cross-session resume. No stage needs to finish in one sitting, which is what makes a ~ 20 h evaluation tractable on free, time-limited GPUs.

13 Future work

(1) Real-environment confirmation of the local/global planner ranking in MuJoCo PointMaze (the offline metric of §11 judges distances with the global model itself); (2) seed replication of the low-data pair and the local/global run; (3) an A100 stride-1 full-data training to replace the stride-2 lower bound; (4) PushT, blocked upstream on a video-backed latent cache (the PointMaze latent path does not transfer directly); and (5) for the local/global track, FOG-style online surrogate fine-tuning on global-model rollouts to keep local gradients on-distribution, plus the built-in projection/context/optimizer ablations — both motivated by the low refinement-acceptance rate of §11.

Reproducibility

Code lives in the `wm-prediction` repository (`src/wm_poc/dino_wm`, `scripts/dino_wm`, `configs/dino_wm`); notebook 02 runs the full pipeline on Colab (T4 or A100) with self-gating, resumable stages, and notebook 07 renders the results. The latent-cache design is documented in `docs/dino_wm_latent_cache_traini`; the figures in this report are generated from the recorded planner telemetry (`reports/pointmaze_planning_log`).

by `reports/make_planning_figures.py`. The two training-curve figures are produced on Drive by notebook 07 (`_summary/`) and copied into `reports/figures/`.

References

- [1] G. Zhou, H. Pan, Y. LeCun, L. Pinto. *DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning*. arXiv:2411.04983, 2024.
- [2] M. Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. arXiv:2304.07193, 2023.
- [3] D. Hafner et al. *Learning Latent Dynamics for Planning from Pixels*. ICML, 2019.
- [4] D. Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. ICLR, 2020.
- [5] N. Hansen, H. Su, X. Wang. *TD-MPC2: Scalable, Robust World Models for Continuous Control*. ICLR, 2024.
- [6] M. Assran et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. 2025.
- [7] P. Amigo, R. Khorrambakht, N. Mansard, L. Righetti. *Coupled Local and Global World Models for Efficient First-Order Model-Based Reinforcement Learning*. arXiv:2602.06219, 2026.
- [8] P.-T. de Boer, D. P. Kroese, S. Mannor, R. Y. Rubinstein. *A Tutorial on the Cross-Entropy Method*. Annals of Operations Research, 134(1):19–67, 2005.